

Day3 实训手册：高频点赞 —— Redis 解决高并发写

一、今日任务目标

目标	说明
理解高频写的压力来源	一次点赞 = 校验 + 查重 + 插入 + 更新计数, 4 条 SQL 全部同步写 MySQL
建立一致性分级思维	区分强一致与最终一致, 理解 BASE 思想
掌握 Redis 承接高频状态操作	Set 存点赞关系、INCR 存点赞数
理解最终一致性	Redis 先写, 数据库稍后由 Day6 的 MQ 补偿
完成改造	点赞/取消点赞、收藏/取消收藏全部走 Redis
课堂实战	独立开发“分享”功能 (第八节, 课内启动、课下完成, 后续几天持续升级)

二、问题场景：压力到底从哪来（先复现，再优化）

2.1 先看基线代码：一次点赞干了什么

day1 基线的 `InteractService.like()` 是“教科书式”的同步写库实现，把它逐行拆开看：

```
1 public Result<Void> like(Long noteId, Long userId) {
2     // SQL ①: 校验笔记是否存在
3     if (noteMapper.selectById(noteId) == null) {
4         return Result.fail(404, "笔记不存在");
5     }
6     // SQL ②: 先查—是否已经点过赞 (注意：并发下“先查后写”有竞态窗口)
7     Long count = likeMapper.selectCount(
8         new LambdaQueryWrapper<NoteLike>()
9             .eq(NoteLike::getUserId, userId)
10            .eq(NoteLike::getNoteId, noteId));
11     if (count > 0) {
12         return Result.fail("您已经点过赞了");
13     }
14     try {
15         // SQL ③: 插入点赞关系记录 (唯一索引兜底拦截重复)
16         likeMapper.insert(new NoteLike(userId, noteId));
17     } catch (DuplicateKeyException e) {
18         return Result.fail("请勿重复点赞");
19     }
20     // SQL ④: 更新笔记行的计数列 (热点行更新!)
21     noteMapper.update(null, new LambdaUpdateWrapper<Note>()
22         .eq(Note::getId, noteId)
23         .setSql("like_count = like_count + 1"));
24     return Result.ok();
25 }
```

也就是说，**用户轻轻点一下赞，后端要同步执行 4 条 SQL**（1 校验 + 1 查重 + 1 插入 + 1 计数更新），全部走完才能返回。收藏、取消点赞同理。

这段代码还埋了两个伏笔：SQL ② 的“先查后写”在并发下有竞态窗口（Day4 解决）；点赞记录只落在 MySQL，没有缓冲（Day6 解决）。

2.2 压力三连：放大、热点、串行

把单次开销放到并发场景里，问题会指数级放大：

压力点	发生了什么	后果
写放大	1 次点赞 = 4 条 SQL；5000 个点赞 = 20000 条 SQL 同步执行	数据库连接池被占满，其他接口（首页、搜索）一起变慢
热点行	爆款笔记的 <code>like_count</code> 是同一行，所有点赞都要改它	InnoDB 行锁排队，RT 随并发升高
同步串行	用户必须等 4 条 SQL 全部落盘才能看到“点赞成功”	接口 RT 高，用户感知卡顿
容量上限	MySQL 单机写入通常在数千 TPS 量级	热门事件流量一来就击穿

换个角度理解：点赞这种操作“高频、简单、用户能容忍晚几秒入库”，却让最重的数据库同步硬扛——典型的**杀鸡用牛刀，牛还累死了**

解决思路呼之欲出：把这类写先交给轻快的 Redis，数据库只做最终归宿（第三节展开）。

2.3 复现步骤（压出数据再动手）

1. JMeter 压测 `POST /api/notes/1/like`：并发 200，请求头 `X-User-Id` 用 `${__Random(1,8)}` 随机；
2. 观察 RT 与 MySQL 压力（`docker stats xhs-mysql`），记录优化前数据；
3. 顺便观察后端 SQL 日志：每个请求刷屏 4 条，这正是 2.1 拆解的内容。

三、理论基础：为什么 Redis 能承接高频写

3.1 Redis 与 MySQL 的写性能差距从何而来

维度	MySQL	Redis
存储介质	磁盘（随机写 + 刷盘）	内存（纳秒级）
写一次要做的事	事务日志、索引维护、锁竞争	纯内存数据结构操作
单机写入量级	数千 TPS	约 10 万 QPS
适合的场景	数据的最终归宿	高频读写的中间状态

结论：高频、用户容忍短暂延迟的写（点赞、计数）先落到 Redis，再由 Day6 的 MQ 匀速写回 MySQL —— 这就是“写缓冲”思想。

3.2 数据一致性分级：不是所有数据都要强一致（重点）

什么是“一致性”？ 简单说就是：数据写入之后，任何人任何时刻读到的，是不是“正确”的那份。

围绕这个问题，工程上形成了三个级别，代价从低到高，保障从弱到强：

(1) 强一致 (Strong Consistency) :

写入完成的瞬间，所有人读到的都是新值，任何时刻读都不会看到旧数据。

代价：必须用事务 + 同步等待实现，性能最低。适用于**钱和账**—余额、订单、库存，错一分都是事故。

它背后是 ACID 思想：原子性 (Atomic)、一致性 (Consistent)、隔离性 (Isolated)、持久性 (Durable) 。

(2) 最终一致 (Eventual Consistency) :

写入后允许存在一段“不一致窗口”（可能几秒），但只要没有新写入，所有副本**最终**会收敛到同一个正确值。

代价：需要异步补偿机制（本课是 Day6 的 MQ 落库）。适用于**高频、轻量、可容忍短暂延迟**的状态操作—点赞、收藏、计数、关注。

注意：“最终一致”不等于“可能丢数据”：只要补偿链路可靠（消息不丢 + 消费幂等），数据一条都不会少，只是晚到几秒。

最终一致是 BASE 思想的落脚点：基本可用 (Basically Available) + 软状态 (Soft state) + 最终一致 (Eventually consistent)，它和 ACID 是两个哲学：ACID 宁可拒绝服务也不给错数据，BASE 宁可给“暂时的旧数据”也要保证服务可用。

(3) 可丢失 (Cacheable) :

数据本身有出处，丢了重建即可，任何时刻都不需要保障。

适用于页面缓存、会话验证码、榜单快照。Day2 的笔记详情缓存、Day5 的热榜重建都属于这一级。

一致性光谱（从左到右代价递减、性能递增）：

1	强一致	←————→	可丢失
2	余额/订单/库存	点赞/收藏/计数/关注	页面缓存/验证码/榜单快照
3	t_note 事务表	本课方案：Redis 先写	Day2/Day5 的缓存设计
4		+ Day6 MQ 异步落库	(丢了重建即可)

怎么判断一条数据属于哪一级？问自己三个问题：

丢一条会不会造成资损（会 → 强一致）？

晚几秒到达用户能不能接受（能 → 最终一致）？

丢了能不能重新算出来（能 → 可丢失）？

点赞显然是第二级—这就是本课敢把点赞先交给 Redis 的理论依据。

面试常问：说说强一致与最终一致的区别，以及各自的应用场景。

点赞场景里最终一致的时间线：

1	T0	用户点赞 → Redis 立即写入，页面计数 +1，用户看到“点赞成功”	(基本可用)
2	T0~T2	MQ 队列短暂积压，t_note_like 表还没有这条记录	(软状态)
3	T3	消费者落库，MySQL 出现记录，与 Redis 收敛一致	(最终一致)

课堂讨论：如果 T3 之前服务重启了，这次点赞会丢吗？

(不会—消息持久化 + 手动 ACK，消费失败会重投；重复投递又引出幂等问题，正是 Day4/Day6 的伏笔。)

3.3 动手：Set 与计数器命令（上课必做）

进入容器：`docker exec -it xhs-redis redis-cli`，跟着敲：

```
1 # ① Set: 点赞关系 (元素唯一, 天然防重复)
2 SADD demo:likes "u1" "u2"           # 返回 2 (新增2个)
3 SADD demo:likes "u1"                 # 返回 0 (已存在)
4 SISMEMBER demo:likes "u1"           # 1 = 已点赞
5 SCARD demo:likes                     # 集合大小 = 点赞数来源之一
6 SMEMBERS demo:likes                 # 查看所有点赞用户
7
8 # ② 计数器: INCR / DECR (原子, 高并发不丢数)
9 INCR demo:count                      # 1
10 INCR demo:count                     # 2
11 DECR demo:count                     # 1
12
13 # ③ 清理
14 DEL demo:likes demo:count
```

课堂讨论：为什么不直接用 `SCARD` 当点赞数，还要单独维护一个 `INCR` 计数？
(`SCARD` 是 $O(N)$ 大 Key 时慢；计数与关系分离后，计数可以独立预热/迁移。)

3.4 本课的中间态数据模型（预告全链路）

```
1 今天 (Day3) :   写 → Redis (关系 + 计数), MySQL 暂不动 → 性能达标, 但数据未持久化
2 Day4:          写 → Redis Lua 原子操作 → 解决幂等与竞态
3 Day6:          写 → Redis → MQ → Consumer → MySQL → 最终一致, 闭环完成
```

四、方案设计

4.1 数据结构

数据	Redis 结构	Key	Member/Value
点赞关系	Set	<code>like:{noteId}</code>	userId
点赞数量	String 计数	<code>like:count:{noteId}</code>	数字
收藏关系	Set	<code>favorite:{noteId}</code>	userId
收藏数量	String 计数	<code>favorite:count:{noteId}</code>	数字

4.2 点赞流程

```
1 用户点赞
2   ↓
3 SADD like:{noteId} userId
4   └─ 返回1 (新增成功) → INCR like:count:{noteId} → 成功
5   └─ 返回0 (已存在)  → 提示"已经点过赞"
```

注意：本天的实现先追求性能，"判断+计数"分两条命令，存在原子性问题 —— 这正是 Day4 要解决的。

4.3 读取改造

- 详情页/列表的点赞数：优先读 `like:count:{noteId}`，没有再回退数据库计数；
- 是否已点赞：`SISMEMBER like:{noteId} userId`。

五、编码实现

按 `code/day3/` 目录完成：

序号	文件	操作
1	<code>service/InteractService.java</code>	【替换】like/unlike/favorite/unfavorite 改为 Redis 实现
2	<code>service/NoteService.java</code>	【替换】fillStatus 改读 Redis；计数用 mergeCounts 合并

核心代码（点赞）：

```
1 Boolean added = redisTemplate.opsForSet().add(RedisKeys.like(noteId),
  userId.toString());
2 if (!Boolean.TRUE.equals(added)) {
3     return Result.fail("您已经点过赞了");
4 }
5 redisTemplate.opsForValue().increment(RedisKeys.likeCount(noteId));
```

六、验证与压测

6.1 功能验证

1. 前端点赞/取消点赞，页面数字即时变化；
2. `SMEMBERS like:1` 能看到点赞用户；`GET like:count:1` 与页面一致；
3. 查看 MySQL：`SELECT COUNT(*) FROM t_note_like` 暂时**不变**（落库交给 Day6，这是预期行为）。

6.2 压测对比

条件：`POST /api/notes/1/like`，并发 200，持续 60 秒，随机用户。

指标	优化前	优化后	变化
QPS			
平均 RT(ms)			
P95(ms)			
MySQL CPU 峰值			
错误率			

七、结果记录表

在第六节表格基础上，补充一句话结论：

优化后点赞请求不再访问 MySQL，接口 RT 从 ms 降到 ms。

八、课堂实战：独立开发“分享”功能（自己动手）

今天的点赞/收藏是老师带着做的，现在轮到你独立开发一个全新功能——分享。
它会在后面几天持续升级：Day4 给它加幂等，Day6 给它异步落库。

需求：新增分享接口 `POST /api/notes/{id}/share`（取消分享：`DELETE /api/notes/{id}/share`）：

- 不需要建表，全部写 Redis（落库交给 Day6）；
- 分享关系用 Set：`share:{noteId}`，成员是 `userId`；
- 分享数量用计数器：`share:count:{noteId}`；
- 重复分享提示“您已经分享过了”，不重复计数。

实现步骤：

1. `RedisKeys.java` 新增 `share(noteId)` 与 `shareCount(noteId)` 两个 Key 方法；
2. `InteractService` 新增 `share(noteId, userId)` 方法，完全仿照 `like` 的写法（`SADD` → 判返回值 → `INCR`）；
3. `InteractController` 新增两个接口（仿照 `like / unlike` 的注解写法）；
4. （选做）在 `NoteService.mergeCounts` 中把分享数合并进详情展示。

关键代码提示（service 层）：

```
1 public Result<Void> share(Long noteId, Long userId) {
2     Boolean added = redisTemplate.opsForSet()
3         .add(RedisKeys.share(noteId), userId.toString());
4     if (!Boolean.TRUE.equals(added)) {
5         return Result.fail("您已经分享过了");
6     }
7     redisTemplate.opsForValue().increment(RedisKeys.shareCount(noteId));
8     return Result.ok();
9 }
```

验收标准：

用 curl/Postman 调用分享接口返回成功，重复调用返回“您已经分享过了”；

`SMEMBERS share:1` 能看到用户，`GET share:count:1` 与分享次数一致；

取消分享后计数减 1，且不会减成负数；

全程没有任何 MySQL 写入（这是今天的预期行为）。

选做挑战：给分享数也加上“读时优先取 Redis、没有再回退数据库”的逻辑，想想和 mergeCounts 像不像？

九、思考题

1. `SADD` 返回 0 和返回 1 分别代表什么？这个返回值能不能用来做幂等？
2. 现在“判断 + 计数”是两条命令，高并发下会不会出现计数和关系不一致？怎么保证原子性？（Day4 预告）
3. Redis 里的数据如果宕机丢了怎么办？（Day6 的 MQ 落库是答案的一部分）